

Regularising a Convolutional Neural Network on Fashion-MNIST

Deep Learning Systems Analysis Report

Dennis O'Higgins

October 2026

1. Report Overview

This report addresses ten-class image classification: assigning each greyscale photograph of a clothing item to a category. It uses the Fashion-MNIST dataset of 70,000 real product images and a convolutional neural network (CNN) built in PyTorch. A baseline CNN is compared with a version that adds dropout and L2 weight decay as a single regularisation change, to test whether it improves performance on unseen images (Srivastava et al., 2014).

2. Dataset and Task Description

I used Fashion-MNIST, a dataset of 70,000 real product images from Zalando, each 28 by 28 pixels in greyscale, divided into 60,000 training and 10,000 test images across ten classes (Xiao et al., 2017). It is published under the MIT licence at <https://github.com/zalandoresearch/fashion-mnist>. The classes are perfectly balanced, with 6,000 training and 1,000 test images each, so accuracy is a fair headline metric. The data are not synthetic, and they were not used in my earlier projects. Pixel values range from 0 to 255. The notebook downloads the four official files and verifies each against the published MD5 checksum before use.

Data preparation.

I scaled pixel values to the range 0 to 1 and standardised them with the mean (0.2862) and standard deviation (0.3532) calculated from the training images only, so that no information from the validation or test images enters training. I then added a channel dimension so that each image has shape 1 by 28 by 28. The 60,000 training images were split in a stratified 90/10 ratio into 54,000 for training and 6,000 for validation, preserving class balance. The validation set guides decisions and the official 10,000-image test set was used once, at the end, to estimate performance on unseen data. Batches of 128 images were drawn with a seeded generator so that both models see the same data in the same order.

3. Model Architecture and Design Decisions

The baseline is a small CNN with 421,642 trainable parameters. Two convolutional blocks, each a 3 by 3 convolution followed by a ReLU activation and 2 by 2 max pooling (with 32 and then 64 filters), extract visual features. A fully connected layer of 128 units and a final layer of 10 units produce class scores. I trained it with the cross-entropy loss and the Adam optimiser (Kingma & Ba, 2015) at a learning rate of 0.001 for six epochs, implemented in PyTorch (Paszke et al., 2019). Six epochs reflects the limits of a CPU-only environment and is itself a limitation discussed below. The baseline contains no regularisation.

Each design choice has a reason. Convolutional layers share weights across the image, so they detect local patterns such as edges and textures with far fewer parameters than a fully connected network,

which suits images. Two convolution blocks are enough for 28 by 28 inputs, and doubling the filters from 32 to 64 lets the second block combine simple features into more complex shapes after pooling has halved the image size. ReLU activations are cheap to compute and avoid the vanishing gradients of saturating functions. Max pooling adds small tolerance to shifts in position and reduces computation. The output layer has ten units, one per class, and returns raw scores that the cross-entropy loss converts to probabilities. Adam was chosen because it adapts the step size for each parameter and works well with little tuning (Kingma & Ba, 2015).

4. Experimental Comparison

The single experimental change is the regularisation strategy. Dropout randomly switches off a share of units during training, which discourages units from depending on one another (Srivastava et al., 2014), and I used a rate of 0.3 after each pooling stage and the hidden layer. Weight decay penalises large weights and is applied here as an L2 penalty of 0.0001 through the optimiser. Loshchilov and Hutter (2019) show that L2 regularisation and weight decay are equivalent for plain stochastic gradient descent but differ for adaptive optimisers such as Adam, so I describe the setting as L2 weight decay in the form PyTorch's Adam implements. Both techniques serve the same purpose, limiting model complexity, so I treat them as one regularisation strategy, and I note that they are not isolated from each other in this design. Everything else was held constant: architecture, optimiser, learning rate, batch size, random seed, data order and number of epochs. This is a meaningful comparison because the baseline already shows a gap between training and validation accuracy, so a technique designed to reduce overfitting has something to act on, and because holding every other setting fixed means any difference can be attributed to the change. To check that the result was not an accident of one random seed, I repeated both models with two further seeds.

5. Results and Interpretation

Table 1 and Figures 1 and 2 summarise the comparison. Regularisation reduced the generalisation gap, the difference between training and validation accuracy, from 0.0214 to 0.0086. It did not raise held-out performance: test accuracy was 0.9146 for the baseline and 0.9036 for the regularised model, and test macro F1 was 0.9143 and 0.9029. Across three seeds the baseline averaged 0.9139 test accuracy (standard deviation 0.0008) and the regularised model 0.9075 (standard deviation 0.0036), with the baseline ahead in all three pairs. Training curves show the regularised model still improving at the final epoch, so six epochs favours the model that learns faster.

Measure	Baseline	Regularised
Train accuracy	0.9454	0.9252
Validation accuracy	0.9240	0.9167
Test accuracy	0.9146	0.9036
Test macro F1	0.9143	0.9029
Generalisation gap	0.0214	0.0086
Test accuracy, mean of 3 seeds	0.9139	0.9075

Table 1. Baseline and regularised model on Fashion-MNIST.

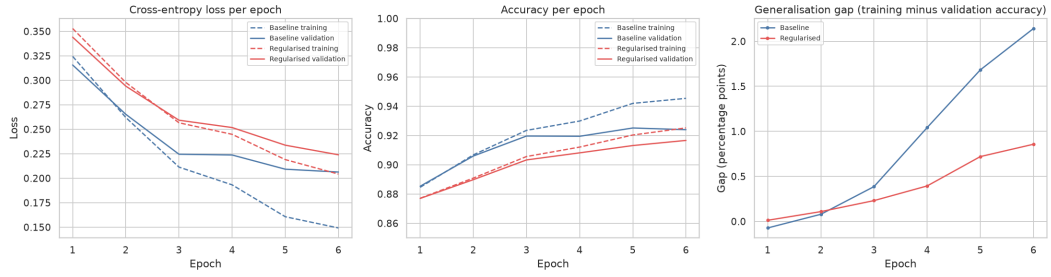


Figure 1. Training and validation loss and accuracy by epoch.

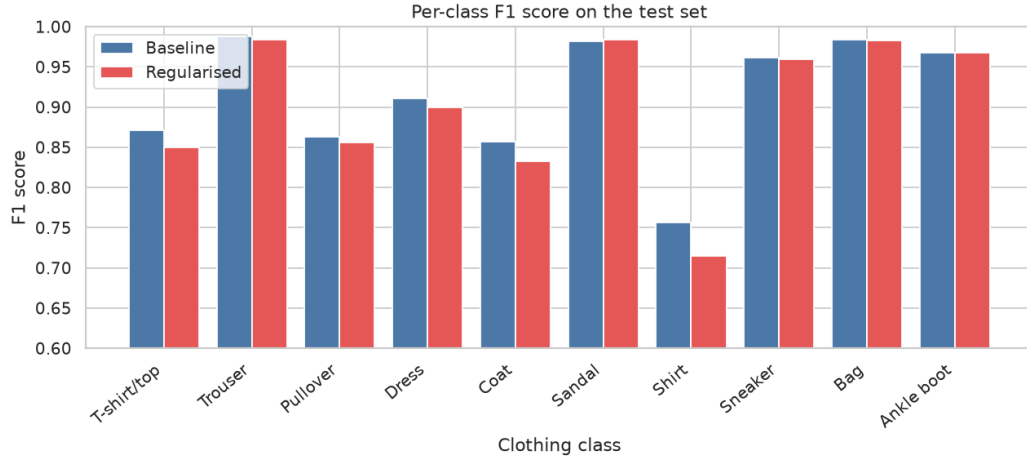


Figure 2. Test F1 score by class for both models.

Error analysis.

The per-class results show where the cost of regularisation fell. Trouser, Sandal, Sneaker, Bag and Ankle boot changed by no more than 0.005 in F1. The upper-body garments lost most: Shirt fell by 0.041, Coat by 0.025 and T-shirt/top by 0.021. As a concrete example, the most frequent error of the baseline was a Shirt predicted as a T-shirt/top (101 of 1,000 shirts), while the regularised model made the same error 156 times. Of the 10,000 test images, 675 were wrong in both models, 179 only in the baseline and 289 only in the regularised model. The confusions occur between garments with similar outlines, and at 28 by 28 pixels the cues that separate a shirt from a T-shirt, such as buttons or collars, are only a few pixels wide. Regularisation did not repair these hard cases. It made the model less able to use fine detail on them.

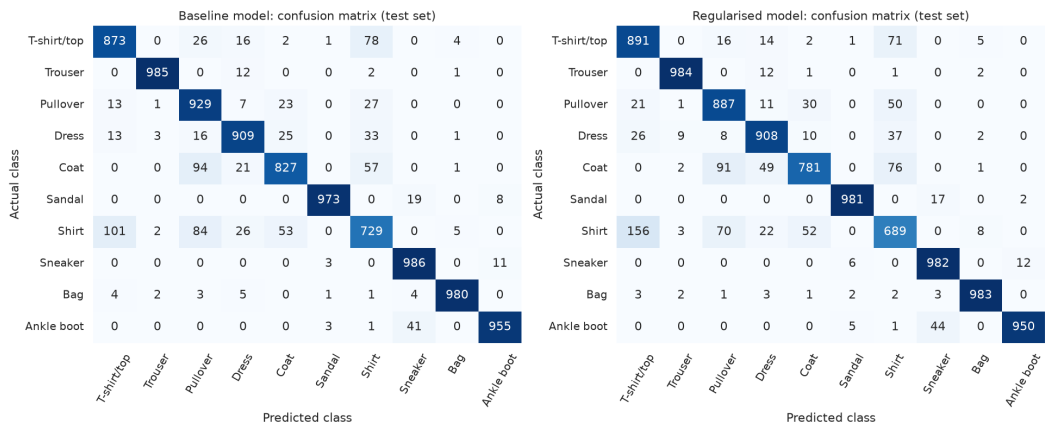


Figure 3. Confusion matrices on the test set.



Figure 4. Examples of misclassified test images.

6. Limitations and Risks

The main limitation of the training setup is the budget: six epochs on a CPU, with both models' validation curves still improving at the end. The regularised model may overtake the baseline with longer training, which I did not test. Dropout and weight decay were changed together, so I cannot say which one caused the observed effect. Three seeds give only a rough estimate of variation (the regularised model's standard deviation in test accuracy was 0.0036), and I did not tune the dropout rate or decay strength. The dataset has its own limits: the images are 28 by 28 pixels, which removes the fine detail needed to separate shirts from T-shirts, and they are centred items on plain backgrounds, so results will not transfer to photographs taken in everyday conditions (Xiao et al., 2017). The evaluation uses one fixed test set, so it cannot show how the models behave under changes in lighting, angle or background.

7. Ethical and Responsible Use

Fashion-MNIST contains product photographs, not people, so it avoids most privacy concerns, but a plausible misuse risk remains: a model with a high benchmark score could be treated as ready for a real retail or cataloguing system although it has only seen curated images from one retailer's catalogue. Uneven performance across classes is also a form of bias. Here the weakest class, Shirt, is served worst by the regularised model, and in a system that treated categories unequally, for example in automatic product tagging for sellers who mostly stock shirts, the same pattern could disadvantage some users. Buolamwini and Gebru (2018) show that aggregate accuracy can hide large differences between subgroups, which is why I report per-class results and not accuracy alone.

8. Future Improvements

Given more time and computing resources, the first step is to train until validation loss stops improving, using early stopping, so that the two models are compared at their best and not at a fixed short budget. Dropout and weight decay should then be varied separately, over a small grid of rates, to identify which one matters. Repeating the comparison over at least ten seeds would give usable confidence intervals. Data augmentation, such as small shifts and flips, addresses overfitting from a

different direction and may help the confusable upper-body classes. A decoupled form of weight decay (Loshchilov & Hutter, 2019) is worth comparing with the standard form used here. Finally, evaluating on a harder, more varied image set would test whether any gains transfer.

9. References

- Buolamwini, J., & Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. *Proceedings of Machine Learning Research*, 81, 77-91. <https://proceedings.mlr.press/v81/buolamwini18a.html>
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *3rd International Conference on Learning Representations (ICLR 2015)*. <https://arxiv.org/abs/1412.6980>
- Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *7th International Conference on Learning Representations (ICLR 2019)*. <https://arxiv.org/abs/1711.05101>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32. <https://arxiv.org/abs/1912.01703>
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56), 1929-1958. <https://jmlr.org/papers/v15/srivastava14a.html>
- Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. *arXiv*. <https://arxiv.org/abs/1708.07747>